Dimensional modelling and ETL exercise

Assignment 1 focuses on dimensional modelling and ETL implementation. Therefore, this assignment is a “hands-on” exercise where you will formulate business questions, create a “classic” Star schema for the company, and develop ETL processes to load data from an OLTP snapshot into a Star schema.

Table of Contents

Preface 2
1. Suggested workflow. 3
2. Business Case: International Air Quality Monitoring Agency 4
3. Dimensional Modelling requirements 7
3.1. Additional Tables (Table_X and Table_Y) 7
3.2. Star Schema 7
3.3. Fact Tables 7
3.4. Dimensions 8
3.5. Metadata 8
4. ETL requirements 9
4.1. Create database 9
4.2. Staging schema 9
4.3. Star schema 9
4.4. ETL pipeline 9
4.5. Validation and Sanity Checks (in Star schema) 9
4.6. Lightweight provenance (PROV-O) 10
5. Report Structure 11
5.1. Synthetic tables (Table_X and Table_Y) 11
5.2. Business/analytic questions 11
5.3. Star schema diagram 11
5.4. Fact tables 11
5.5. Dimension tables 12
5.6. Snowflake vs. Star 12
5.7. ETL and validation summary 12
5.8. Reflection and lessons learned 12
6. What to Submit 13

Preface

Deadline: Make sure to upload all your results **by November 4, 2025.**

Submission guidelines: We are going to follow a few naming conventions throughout the assignment. Please **pay attention to them and follow carefully**, as this will ensure your code executes correctly in the test environment.

Compress your solution directory and upload it to TUWEL using the following filename:

`BI_2025_Assignment_1_Group_xxx.zip/.tgz`

Within this ZIP-file, the top-level directory is named BI_Projects. Inside this directory, there is a folder named DWH1_**xxx**, **please replace <xxx> with your actual Three-Digit Group Number**, (with the leading zeros if necessary).

This DWH1_xxx folder should contain all your deliverables as follows:

```
BI_Projects
  \-DWH1_xxx
    |
    +---csv
    |       15 original CSV files
    +---data
    |       tb_<Table_X>.csv, tb_<Table_Y>.csv
    +---ddl
    |       airq_create_dwh_xxx.sql, airq_create_stg_xxx.sql, airq_create_ext_xxx.sql
    |       airq_reset_dwh_xxx.sql, airq_reset_stg_xxx.sql,
    +---etl
    |       a1_etl01_dim_timeday.sql
    |       a1_etl02_dim_<meaningfulname1>.sql
    |       a1_etl03_dim_<meaningfulname2>.sql
    |       a1_etl04_dim_<meaningfulname3>.sql
    |       ...
    |       a1_etl07_ft_<meaningfulname1>.sql
    |       a1_etl08_ft_<meaningfulname2>.sql
    +---post
    |       a1_check01_ri.sql
    |       a1_check02_scdranges.sql
    |       a1_check03_counts.sql
    |       ...
    +---prov
    |       prov_airq_dwh_xxx.jsonld
    +---sqldump
    |       sqldump_airq_dwh_xxx.sql
    |   AirQ_Part1_xxx.ipynb
    |   AirQ_ERD_dwh_xxx.png (or .pdf)
    |   group_xxx.txt
    |   Report_Part1_Group_xxx.pdf


* Please replace xxx with Your Group Number everywhere
```

Each group member should upload a submission. In the best-case scenario, when members of a group work on the assignment together as a team, then for the two members of the same group, the contents of their submission will be identical.

Questions. Please post general questions and discuss issues in the TUWEL discussion forum. We appreciate it if you help other students (and may take this into account if you are short a few points for a better grade). For obvious reasons, however, please do not post any solutions in this forum.

For specific questions regarding the assignments, you can contact our tutors

- Ildar Fatkullin (ildar.fatkullin@tuwien.ac.at)
- Bosse Behrens (bosse.behrens@tuwien.ac.at)
- Alwin Krycha-Weilinger (alwin.krycha-weilinger@tuwien.ac.at)

Or in case of other issues

- Katja Hose (katja.hose@tuwien.ac.at).

1. Suggested workflow.

To structure your work, we recommend the following sequence of steps:

1) Team setup

Assign roles (randomly) within your group (**Student A** and **Student B**). Each student will take primary responsibility for specific portions of the project while collaborating on the overall design.

2) Understand the case

Carefully read the business case description to understand the monitoring system, its processes, and the analytical objectives.

3) Review the OLTP snapshot

Study the provided schema and CSV files. Familiarise yourself with entities, relationships, and data characteristics through exploratory SQL queries.

4) Define extensions

Design two additional tables (Table_X and Table_Y) to extend the OLTP snapshot, following the established naming and structural conventions.

5) Formulate business questions

Each student should propose 4–6 meaningful business questions that the OLAP system should be able to answer. These questions will guide your schema design.

6) Design the star schema

Model a dimensional schema that supports your business questions. Ensure compliance with all requirements for fact tables, dimensions, hierarchies, measures, etc.

7) Implement ETL

Implement ETL process from the OLTP snapshot to your OLAP schema using the SQL-first approach (all individual steps are SQL statements to be executed by the database engine).

8) Validate

Validate the correctness of your ETL with 5–7 SQL queries against your star schema. Each query must return a result and include a one- or two-sentence interpretation.

9) Prepare deliverables

Compile all required artefacts — report, schema diagram, database dump, and ETL scripts — as described in the submission guidelines.

2. Business Case: International Air Quality Monitoring Agency

You are joining a team of developers supporting an international environmental agency that runs an air-quality monitoring programme. The agency currently has air-monitoring sensor devices deployed in urban and industrial locations across 36 cities in 20 countries.

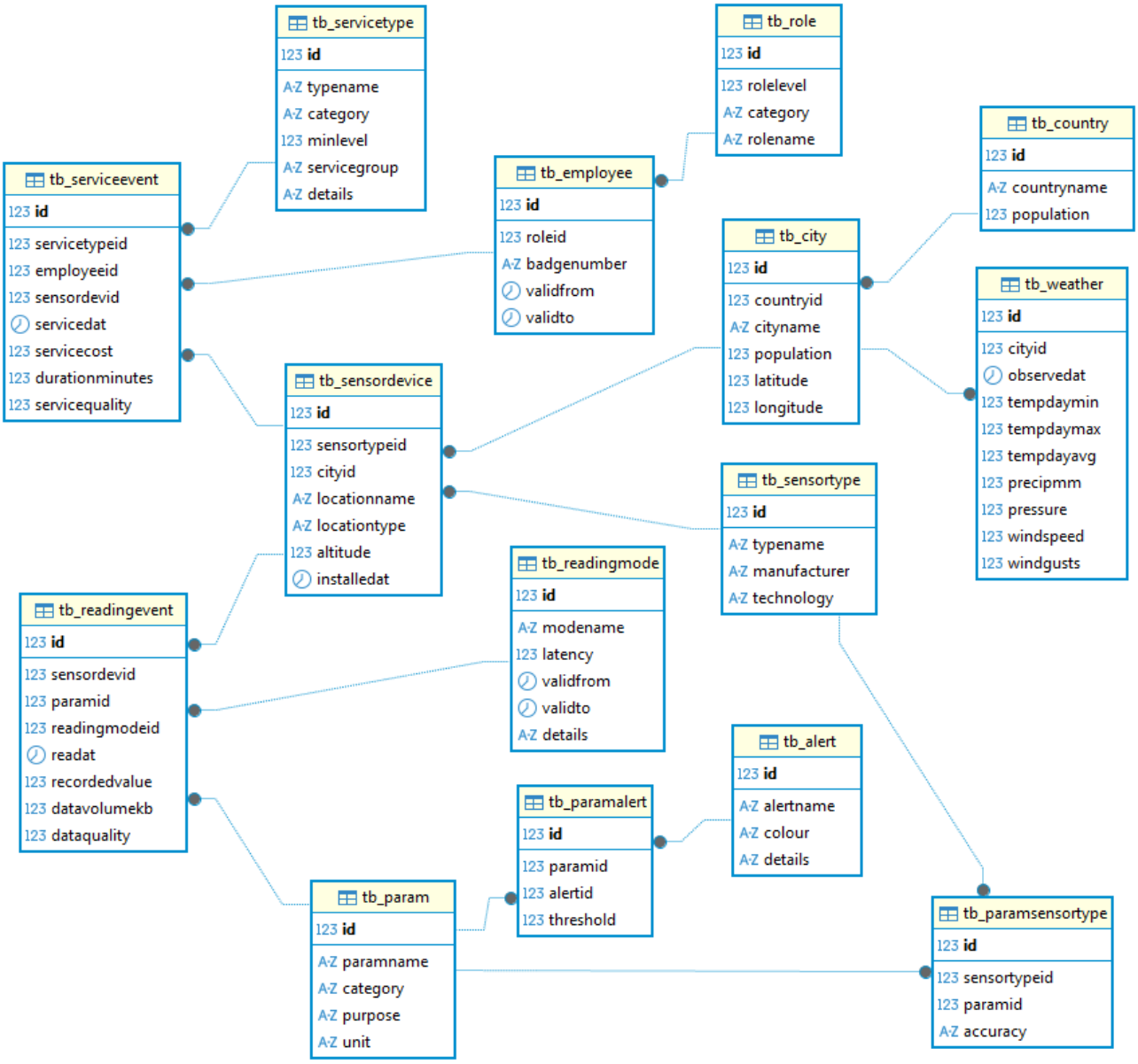

Figure 1. OLTP snapshot

Each device is physically installed at a specific location, described by type (urban, suburban, industrial, etc.), altitude, and installation date. Every device belongs to a sensor type — defined by its manufacturer and sensing technology (optical, electrochemical, laser, etc.).

Sensor types differ in their capabilities - each can measure only a subset of the agency's environmental parameters, such as particulate fractions, gases, heavy metals, etc. These capabilities are managed centrally. In practice, this means there is a many-to-many relationship between sensor types and parameters, with a stated accuracy (high/medium/low) for each allowed pairing.

In addition to pollutant readings, the agency records daily weather observations in every city. Each observation includes minimum, maximum, and average temperature, precipitation totals, mean sea-level pressure, and wind statistics (average speed and peak gusts). Weather is collected once per city per day and synchronised to the same calendar as pollutant readings, enabling analysts to link air quality behaviour with meteorological conditions.

Every environmental measurement is logged as a reading event - a device records a parameter on a given date, under a particular reading mode. Reading modes (Rapid, Low Power, Standard, etc.) are time-bound — they have effective start and end dates, since new modes may be introduced or old ones retired. Each reading captures the measured value plus two operational metrics - data volume (the telemetry transmitted) and a data quality score (1..5). These make the readings useful not only for environmental analysis but also for operational monitoring of telemetry load and reliability.

The agency enforces a uniform alert ladder across pollutants - Yellow, Orange, Red, and Crimson. For each parameter, policy teams define thresholds for these levels. Thresholds differ by parameter (e.g., particulate matter vs. gases vs. biological markers), but the semantics are consistent. By convention, only the highest level exceeded is considered for any given reading — if a value crosses Red, it is classified as Red, not as Yellow or Orange as well. Analysts frequently aggregate exceedances by city, device, parameter, date, and alert level.

Maintaining this infrastructure requires regular servicing. The agency defines a catalogue of service types — calibration tasks, software actions, diagnostic procedures, hardware interventions — each with a category, a minimum qualification level (Entry, Junior, Senior, Lead), a service group (to cluster related tasks), and concise procedural details.

Technicians are identified by badge number. Over time, each technician holds a succession of roles with valid-from and valid-to dates, as people are promoted, reassigned, or retrained. Each role has a category and a level. When a service event occurs, the system records which technician performed which service on which device, when it was done, and its operational metrics - cost, duration, and service quality (1–5).

A critical nuance is that the minimum qualification level is not strictly enforced. While policy prescribes the required level for each service type, there are rare cases where a technician performs a service despite holding a lower role level at the time. These cases remain part of the dataset and provide analysts with the opportunity to check compliance and study the circumstances of underqualified assignments.

This system is designed to serve multiple audiences:

- Public officials monitor trends, hotspots, and compliance across cities and countries.
- Managers track data volumes, quality, and service activity to optimise uptime and cost.
- Policy analysts rely on published alert thresholds to compare exceedances across pollutants.
- Engineering leads examine device reliability by manufacturer and technology, checking whether certain sensor models underperform in specific contexts.

Each group must also extend the OLTP snapshot with two additional tables:

- Table_X: an entity table with at least 6 (six) rows, following the structural conventions of the snapshot.
- Table_Y: a bridge table that creates a many-to-many relationship between Table_X and one existing table in the OLTP snapshot. It should contain at least 12 (twelve) rows and may include additional metadata columns.

Information from at least one of these tables must be meaningfully integrated into your dimensional OLAP schema. How you design and justify this integration is up to your creativity.

A few further modelling aspects are directly visible in the schema you will use:

- The "country → city → device" chain forms a shared geographic axis across both readings and service activity.
- Sensor capability is many-to-many - a pollutant can be measurable by multiple models, and a sensor type can measure multiple pollutants.
- Technician roles change over time, and service assignments must be validated against the technician's role and level at the service date.
- Reading modes evolve with validity dates.
- Alert thresholds are parameter-specific, so cross-parameter comparisons must be framed in terms of exceeded levels, not raw values.

Your task is to design a portion of a data warehouse — a data mart — that supports decision-making across this ecosystem. The schema you design should enable insights into environmental trends, device performance, technician activity, service effectiveness, and city-level conditions.

When designing your schema, consider:

- Shared (conformed) dimensions such as geography or time, used across multiple facts.
- Opportunities for Slowly Changing Dimensions (SCD), particularly Type 2, where historical tracking is relevant.
- The choice of time dimensions, including whether one or two granularities (e.g., day vs. month) are appropriate/necessary.
- Justification for any degenerate or junk dimensions.

3. Dimensional Modelling requirements

3.1. Additional Tables (Table_X and Table_Y)

You must extend the OLTP snapshot with two manually created tables, meeting the following conditions:

- Table_X: an entity table with a surrogate primary key (id) and at least one additional column of any data type. It must contain at least 6 (six) rows.
- Table_Y: a bridge table that creates a many-to-many relationship between Table_X and one existing OLTP table. It must include foreign keys to both sides of the relationship, and may include optional metadata columns (e.g., priority, date, level). It must contain at least 12 (twelve) rows.
- Both tables must be created manually as CSV files and then loaded into your schema.
- Data from at least one of these tables must be meaningfully integrated into your star schema (e.g., as a dimension, as part of a hierarchy, or in another justified role).
- In the report, name the existing OLTP table that Table_Y bridges to, and justify that choice.

3.2. Star Schema

Your OLAP schema must be a Star schema. Snowflake schemas are not allowed, and their usage may result in a point deduction.

3.3. Fact Tables

You must design exactly two fact tables, named ft_<meaningfulname>, meeting the following conditions:

- At least three measures per fact table (healthy range is 3–6).
- At least one measure per fact table must be fully additive across time and other dimensions (e.g., counts, costs, volumes).
- Semi-additive measures are optional but welcome (e.g., averages, ratios, scores).

In the report, clearly identify ownership/responsibility:

Student A: Fact 1 (responsibility split: Student A ≈ 80%, Student B ≈ 20%)

Student B: Fact 2 (responsibility split: Student B ≈ 80%, Student A ≈ 20%).

Primary responsibility (80%) means that the designated student leads the design and documentation of that fact table (grain, measures, dimensions, and business questions) and is mainly accountable for its correctness. The other student still carries secondary responsibility (about 20%) — problems found in that fact table will reflect partly on both, though primarily on the designated owner.

Both students are expected to collaborate closely on all fact tables - discussing design choices, reviewing each other's work, and providing feedback on grains, measures, dimensional links, and business questions.

The OLTP snapshot includes clear candidates for fact tables, but you are encouraged to think creatively. Derived or less obvious fact tables are welcome if they enable more interesting or meaningful analytical queries.

3.4. Dimensions

You must design at least four non-time dimension tables, named dim_<meaningfulname>, meeting the following conditions:

- At least one conformed (shared) dimension used by both fact tables (besides time).
- At least two dimensions must include three levels of hierarchy each (and each fact must use at least one such dimension).
- At most one Slowly Changing Dimension of Type 2 (SCD2).
- At most one degenerate dimension is allowed (justify in 1–2 sentences).
- At most one junk dimension is allowed (counts as a single dimension even if it consolidates multiple attributes).

You must also create one or two time dimensions, named dim_time<granularity>, where <granularity> is replaced with actual granularity, for example, dim_timeday or dim_timemonth, etc. If you create two, they must have different granularity.

3.5. Metadata

Include an audit column:

- etl_load_timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP

in every dimension and fact table to mimic production load tracking.

4. ETL requirements

The ETL process (Extract–Transform–Load) is a core part of this assignment. Your Jupyter notebook must be fully reproducible - it should run from start to finish without manual intervention.

4.1. Create database

Create database airq. Inside it, create two schemas

- stg_xxx (staging schema)
- dwh_xxx (star schema).

All staging tables live in stg_xxx. All dimensional/fact tables live in dwh_xxx.

4.2. Staging schema

Create the staging tables, your additional Tables X and Y, load all provided CSV files into stg_xxx.

Note: The process that produced the given DDL script (airq_create_stg_xxx.sql) did not run flawlessly.

As a result, while the individual CREATE TABLE statements in that script are correct, they are not in the correct dependency order. Reorder statements to satisfy foreign keys before execution. CSV loads are implemented via Pandas .to_sql().

4.3. Star schema

Create your dimensional/fact tables by executing appropriate DDL statements. Create dimensions first, then facts. Include an audit column (etl_load_timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP) in every dimension and fact table to mimic production load tracking.

Do not create tables manually outside the ETL - table creation must be part of your pipeline.

4.4. ETL pipeline

Create an ETL pipeline to load data from stg_xxx to dwh_xxx.

SQL-Based ETL (SQL-first)

- Write SQL scripts that extract data from stg_xxx, transform it, and load it into dwh_xxx (joins, filters, lookups, SCD logic, etc.).
- Execute these ETL scripts in the Jupyter Notebook (via SQLAlchemy).

4.5. Validation and Sanity Checks (in Star schema)

Once your data is loaded into the OLAP schema, you need to show that everything landed correctly. Validation is an important required step and should appear as a clearly marked section at the end of your notebook. All checks must be SQL queries executed against the final star schema dwh_xxx.

We don't give you an exact checklist here - you should decide what to test. As a general rule, aim for 5–7 SQL queries that collectively verify the data appears correct. Each check must show the query result and a 1–2 sentence interpretation.

Here are some ideas you might cover (pick a mix that makes sense for your model):

- Row counts: check expected magnitudes in facts and dimensions (e.g., counts per device, per day, or per city).
- Referential integrity: count fact rows with foreign keys not found in dimensions (should be zero).

- Key uniqueness: check for duplicates on surrogate keys (should be zero).
- Confirm that rows from Table_X/Table_Y are represented in the star schema as intended.
- SCD Type 2 consistency: make sure no overlapping validity ranges for the same key, and exactly one current row.
- Nulls & domains: check required fields for unexpected NULLs, and verify that enumerated fields only contain allowed values.
- Measure ranges: measures make sense (no negative costs, no quality scores outside 1–5).
- Time coverage: confirm min/max dates are as expected and there are no unexplained gaps.

Each query must return a result and include a concise one- or two-sentence interpretation.

4.6. Lightweight provenance (PROV-O)

After your ETL finishes, run a short "Provenance" cell in the notebook that records "what was used", "what was run", "what was produced", and "who/what did it" - in W3C PROV-O (JSON-LD). This is not a focus of the assignment - it's a light audit trail to help you (and us) see that the lineage makes sense.

1. In the provenance section in `AirQ_Part1_xxx.ipynb` update the simple inputs - student names, fact tables' names, list of OLTP tables you actually used, the target tables created in `dwh_xxx`, etc. Leave the rest of the template as is.
2. Run the cell to produce a JSON-LD structure.
3. Skim & sanity-check the JSON-LD:
   - There is one prov:Activity (your ETL run) with a timestamp.
   - It prov:used your OLTP inputs and ETL SQL scripts, and prov:generated your dimensions/facts.
   - It lists prov:Agent(s) (student names) and prov:SoftwareAgent(s) (PostgreSQL, Jupyter).
   - If present, row counts and any derivations (e.g., a date series) look reasonable.
4. Mention it in your report (1–2 sentences) in section "5.7 ETL and validation summary" if you have comments regarding the provenance cell and/or the resulting JSON-LD.

5. Report Structure

Your report is a central deliverable for Assignment 1. It should describe both your technical implementation and your analytical reasoning. The report must be 2–4 pages of content (hard limit 4 pages), plus a separate cover page with title and names (identify Student A and Student B). The ERD figure and any optional appendix do not count toward the limit. Be concise and to the point - avoid generic definitions; document only design choices, grains, measures, keys, and validation insights.

5.1. Synthetic tables (Table_X and Table_Y)

Describe the purpose and meaning of your two additional tables, describe the fields of both tables, together with data types and row counts. Explain their role in extending the OLTP snapshot, name the OLTP table bridged by Table_Y, and how their data is integrated into your OLAP schema.

5.2. Business/analytic questions

Each student must propose at least four questions (more are welcome). Present them in a clear list format. Questions should be non-trivial, not easily solved in a transactional schema, designed to highlight the strengths of your OLAP schema (e.g., comparisons, breakdowns, trends, KPI monitoring).

Submit your questions as:

Student A:

Q1 ...

Q2 ...

...

Student B:

Q1 ...

Q2 ...

...

5.3. Star schema diagram

Include your star schema as an ER diagram. All fact and dimension tables should be visible, with keys and relationships clearly indicated. This can be done in DBeaver: right-click on the star schema (dwh_xxx) -> View Diagram. Export the ERD as AirQ_ERD_dwh_xxx.png (or .pdf) and include it in your submission.

5.4. Fact tables

Remember that every design choice should be tied back to the team's business questions.

For each fact table:

- State which student was primarily responsible.
- Give a concise business motivation for choosing this fact table.
- Define its grain clearly.
- List all measures with aggregation rules (e.g., SUM, AVG, COUNT).
- List all linked dimensions (which ones are shared across facts).

5.5. Dimension tables

For each dimension table:

- Describe its hierarchy levels (if any).
- Did you create one or two time dimensions? Why one? Why two? Briefly justify your choice.

Describe your SCD Type 2 if implemented. Describe your degenerate and/or junk dimensions if implemented.

5.6. Snowflake vs. Star

You were required to implement a Star schema. Reflect on whether you would have preferred a Snowflake schema for your particular project and business questions if it were allowed. If yes, why? If not, why not? You may answer individually (Student A/Student B) if your opinions differ.

5.7. ETL and validation summary

Summarise your validation checks - what you tested (row counts, referential integrity, SCD consistency, etc.), what were the results, and one or two sentences of interpretation per validation query.

5.8. Reflection and lessons learned

A short section where each student writes 1-2 paragraphs on what they personally learned. Possible themes: dimensional modelling, teamwork, ETL practices, importance of validation, etc. This section is not graded heavily, but it shows understanding beyond implementation.

6. What to Submit

Your submission must include the following deliverables, all packaged in a single ZIP file:

6.1. A concise, well-structured report following the Chapter 5 guidelines.

Be focused: avoid lengthy definitions or generic explanations — clarity and precision are valued. Length: 2–4 pages of content (max 4), plus a cover page; the ERD and optional appendix do not count.

6.2. Jupyter Notebook AirQ_Part1_xxx.ipynb.

The notebook must be fully reproducible - it should run from start to finish without manual interventions.

6.3. DDL scripts

airq_create_stg_**xxx**.sql: correct version respecting dependency order

airq_create_ext_**xxx**.sql: DDL script for Tables X and Y you invented to enrich the OLTP snapshot

airq_create_dwh_**xxx**.sql: drop and recreate tables of your star schema (fact and dimension tables)

airq_reset_stg_**xxx**.sql: drop and recreate your staging schema

airq_reset_dwh_**xxx**.sql: drop and recreate your star schema (fact and dimension tables)

6.4. ETL scripts

Submit your SQL scripts used to load data into dwh_**xxx** and your post-ETL checks.

6.5. Database dump

Submit your populated OLAP schema (dwh_**xxx**) as a SQL dump file. Follow the provided dump instructions to ensure consistency. Make sure the dump includes both schema and data.

6.6. File naming and packaging

All files should be named clearly and logically (use underscores, not spaces).

```
BI_Projects/
  DWH1_xxx/
    csv/       # original CSV files
    data/      # Only "Tables X and Y" CSVs
    ddl/       # DDL only (staging, X/Y, warehouse, optional reset)
    etl/       # SQL-first ETL steps: a1_etl01_...sql, a1_etl02_...sql, ...
    post/      # Post-ETL checks: a1_check01_...sql, a1_check02_...sql, ...
    prov/      # Auto-generated provenance JSON-LD from the notebook (required)
    sqldump/   # Export produced by pg_dump
    AirQ_Part1_xxx.ipynb
    AirQ_ERD_dwh_xxx.png / .pdf
    group_xxx.txt
    Report_Part1_Group_xxx.pdf
```

Replace **xxx** with your three-digit group number. Use ASCII symbols, no spaces/Umlauts/special characters. Place all deliverables in a single folder, compress it into a .zip:

`BI_2025_Assignment_1_Group_xxx.zip/.tgz`

and upload it to TUWEL.